

DeSalut Business OS — Roadmap Implementasi Modul Sales & POS

Sumber: Audit Final — Modul Sales & POS (versi konsolidasi) **Disusun sebagai:** Rangkaian sprint + prompt AI coding assistant yang siap tempel (Claude Code / Cursor / Lovable / Codex)

1. Executive Summary

Audit final menyimpulkan bahwa fondasi data model dan business logic modul Sales & POS sudah kuat, tapi ada **4 celah kritis** yang harus ditutup sebelum lanjut ke fitur besar berikutnya:

1. Data packaging di Shift Management masih manual padahal datanya sudah otomatis tercatat di POS.
2. Permission matrix (`permissions.ts`) belum lengkap dan tidak konsisten dipakai — halaman POS sama sekali tanpa guard berbasis role.
3. Role Production masih punya akses navigasi ke `/sales/pesanan`, bertentangan dengan keputusan bisnis bahwa Production hanya boleh mengakses modul Production & Inventory.
4. (Temuan tambahan pasca-audit) Relasi Queue↔Sale putus untuk transaksi asal Pre Order — `saleId` di entri Queue lama tidak pernah di-relink ke `Sale` baru saat PO di *Complete*, sehingga guard refund bocor total dan status di halaman Pesanan salah tampil "Selesai".

Di atas fondasi itu, ada dua fitur bisnis **High priority** yang sudah dikonfirmasi dibutuhkan tapi belum siap: **DP & Pelunasan** (service layer sudah ada tapi UI belum tersambung, dan ada bug double-charge aktif) dan **Split Payment kombinasi metode bayar** (belum ada sama sekali, butuh perubahan tipe data `Sale.payment`).

Selain itu ada 4 perbaikan **Medium priority** bereffort rendah dengan dampak nyata (tag "SALINAN" di reprint, auto-save cart, blokir produk nonaktif di cart, guard shift ganda), dan satu item **Low priority** (race condition multi-device) yang sengaja **tidak dieksekusi sekarang** — cukup dipantau sampai ada backend dengan session tracking server-side.

Fitur-fitur yang **secara eksplisit dikeluarkan dari roadmap** (produk favorit, diskon, quick order, pajak/NPWP, partial refund per item, halaman Sales History terpisah, dashboard kompleks, halaman Kelola Kategori terpisah) **tidak muncul di sprint manapun** di bawah ini, sesuai keputusan bisnis yang sudah dikonfirmasi di audit.

Roadmap ini terdiri dari **7 sprint**, disusun berdasarkan dependency teknis dan risiko — bukan sekadar urutan priority — supaya setiap sprint berhenti di titik yang stabil dan production-ready sebelum sprint berikutnya dimulai.

2. Implementation Roadmap (Ringkasan)

#	Sprint	Audit Ref	Priority Asli	Kompleksitas
1	Permission Matrix & Route Guard	#2	Critical	Medium
2	Packaging Auto-Prefill Shift ↔ POS	#1	Critical	Medium
3	Relink Queue ↔ Sale (Fix Guard Refund Bocor)	#3B	Critical	Medium
4	Koreksi Akses Navigasi Role Production	#3	Critical	Easy-Medium (tergantung opsi)
5	DP & Pelunasan — UI + Fix Bug Double-Charge	#5	High	Medium
6	Edge Case Murah: SALINAN, Auto-save Cart, Blokir Produk Nonaktif, Guard Shift Ganda	#6, #7, #8, #9	Medium	Easy (gabungan)
7	Split Payment — Kombinasi Metode Bayar	#4	High	Medium-Hard

Tidak masuk sprint (sesuai audit):

- #10 Race condition multi-device/multi-tab — diturunkan ke Low, dipantau saja, baru dieksekusi begitu ada backend dengan session tracking.
- Halaman "Kelola Kategori" terpisah — ditunda, bukan dibatalkan; belum ada sprint karena belum ada urgensi operasional.
- Semua item di Bagian 2 audit (produk favorit, diskon, quick order, pajak/NPWP, partial refund per item, Sales History halaman terpisah, dashboard kompleks) — dicabut total dari roadmap.

3. Dependency Explanation

Sprint 1 (Permission Matrix) dan Sprint 2 (Packaging Auto-Prefill) ditaruh paling awal karena audit secara eksplisit menyatakan keduanya adalah *fondasi keamanan dan integritas data* yang **tidak saling bergantung dengan fitur lain**. Sprint 1 dieksekusi lebih dulu karena celah keamanannya (POS tanpa guard sama sekali) berdampak lebih luas — semua sprint berikutnya yang menyentuh POS/Shift/Pesanan sebaiknya berjalan di atas permission matrix yang sudah benar, supaya fitur baru (DP, split payment) langsung lahir dengan guard yang benar, bukan ditambal belakangan.

Sprint 3 (Relink Queue↔Sale) berdiri sendiri karena domain teknisnya berbeda (Pre Order/Queue/Refund), tidak overlap file dengan Sprint 1-2, dan sifatnya adalah *data integrity bug* aktif yang sedang bocor di produksi (guard refund tidak berfungsi untuk semua sale asal Pre Order). Ini harus ditutup sebelum Sprint 5 (DP & Pelunasan) karena Sprint 5 juga menyentuh alur "PO → Complete → Sale baru" — kalau relink Queue belum benar, fitur DP yang baru ditambahkan di titik yang sama berisiko mewarisi bug yang sama atau membuat regression baru lebih sulit dilacak.

Sprint 4 (Navigasi Production) independen secara teknis (hanya menyentuh `navigation.ts` dan mungkin satu halaman baru), tapi **butuh keputusan bisnis dulu** (Opsi A vs Opsi B — lihat Bagian 5 audit). Ditaruh setelah Sprint 1-3 karena efektif tidak ada risiko menunggu, dan kalau Opsi B dipilih (bikin view antrean produksi terpisah), implementasinya akan lebih bersih kalau permission matrix (Sprint 1) sudah final — scope Production ke Production & Inventory saja sudah bisa langsung dipakai sebagai basis guard halaman baru itu.

Sprint 5 (DP & Pelunasan) menyentuh alur PO Complete yang sama dengan Sprint 3, jadi ditaruh tepat setelahnya supaya tidak ada dua tim/dua AI session mengubah `preorder-page.tsx` `complete()` secara bersamaan dengan pemahaman yang berbeda tentang state Queue. Fix bug double-charge di sprint ini bergantung langsung pada `remainingBalance` yang sudah benar dihitung — logic ini sudah ada di service layer, jadi effort utamanya di sisi UI, bukan data model.

Sprint 6 (edge case murah) sengaja ditaruh setelah fondasi kritis selesai tapi **sebelum** split payment penuh, karena semua findingnya (#6-#9) adalah perubahan lokal, low-risk, tidak menyentuh tipe data `Sale.payment` — bisa dikerjakan kapan saja tanpa menunggu Sprint 7, dan audit sendiri menyebut ini "bisa dikerjakan paralel, effort rendah." Ditaruh sebelum Sprint 7 supaya backlog cepat mengecil dan tidak menumpuk di akhir.

Sprint 7 (Split Payment kombinasi metode) sengaja **paling akhir** karena effort-nya paling besar dan mengubah tipe data inti `Sale.payment` dari single value menjadi array `{method, amount}[]` — perubahan ini menyentuh checkout POS, shift closing/rekonsiliasi, receipt, dan analytics sekaligus. Menjalankannya lebih dulu akan memperbesar risiko konflik dengan Sprint 5 (DP, yang juga menyentuh total & payment) dan Sprint 1 (permission guard yang perlu sudah stabil di titik checkout). Sprint ini juga masih punya **open question** dari audit (apakah total gabungan metode harus pas 100% sama dengan tagihan, atau tetap ada logic kembalian khusus di porsi cash) yang harus dijawab sebelum eksekusi — prompt sprint ini akan menyertakan kedua opsi secara eksplisit tanpa mengasumsikan salah satu.

4. Estimated Complexity per Sprint

Sprint	Kompleksitas	Alasan
1 — Permission Matrix	Medium	Menyentuh banyak file (<code>permissions.ts</code> , <code>pos-page.tsx</code> , <code>shifts-page.tsx</code> , layout root) tapi logic-nya

Sprint	Kompleksitas	Alasan
		straightforward — melengkapi matrix & mengganti pengecekan manual jadi terpusat.
2 — Packaging Auto-Prefill	Medium	Butuh agregasi data dari <code>stockMovementService.list()</code> yang difilter by type & rentang waktu, lalu direkonsiliasi dengan draft manual yang sudah ada.
3 — Relink Queue↔Sale	Medium	Perubahan kecil di titik kode (satu langkah tambahan di <code>complete()</code>), tapi butuh testing teliti karena menyangkut guard refund & status pesanan — konsekuensi salah implementasi tinggi.
4 — Navigasi Production	Easy (Opsi A) / Medium (Opsi B)	Opsi A hanya hapus satu item menu. Opsi B butuh halaman baru + guard scope terbatas.
5 — DP & Pelunasan	Medium	Service layer sudah lengkap; kerja utama di UI (dua popup baru) + satu bug fix nilai <code>total</code> .
6 — Edge Case Murah	Easy (gabungan 4 finding kecil)	Masing-masing finding berdiri sendiri, scope kecil, tidak ada perubahan tipe data.
7 — Split Payment Kombinasi	Medium-Hard	Perubahan tipe data inti yang menyentuh banyak titik konsumsi (<code>Sale.payment</code>) — checkout, shift closing, receipt, analytics.

5. Complete Implementation Prompt — Sprint 1

```
# SPRINT 1 — Permission Matrix & Route Guard Hardening

## Sprint Objective
Lengkapi `src/lib/permissions.ts` (MATRIX) supaya seluruh 8 capability memiliki entry eksplisit untuk keempat role (admin, owner, cashier, production), lalu ganti SEMUA pengecekan role manual yang tersebar di `pos-page.tsx` dan `shifts-page.tsx` supaya lewat `usePermission().can(...)` yang membaca dari MATRIX yang sama. Tambahkan satu route guard generik di layout root yang juga membaca dari MATRIX ini. Setelah sprint ini selesai, halaman POS harus punya guard capability berbasis role — saat ini POS sama sekali tidak punya guard apapun.

## Audit Findings Included
- Finding #2 (Critical): "Permission matrix belum lengkap dan tidak konsisten dipakai."
```

Business Rules to Preserve

- Role `admin`: akses penuh ke semua capability (baseline yang sudah ada di MATRIX, jangan diubah).
- Role `owner`: akses read-only ke seluruh modul Sales & POS (bukan admin, tapi bisa melihat semua data yang admin bisa lihat).
- Role `cashier`: akses ke capability operasional harian kasir (POS, Shift, Pesanan) – TIDAK termasuk akses ke Production/Inventory kecuali sudah eksplisit ada di kode sebelumnya.
- Role `production`: scope terbatas HANYA ke modul Production dan Inventory. Role ini TIDAK boleh punya capability apapun yang mengarah ke data Sales/POS/Pesanan/Shift finansial (detail penjelasan lebih lanjut ada di Sprint 4 – di sprint ini, cukup pastikan MATRIX tidak memberi production capability di luar Production & Inventory; JANGAN ubah `navigation.ts` di sprint ini, itu scope Sprint 4).
- Kebijakan refund (void seluruh transaksi, bukan partial per item) tidak berubah oleh perubahan permission ini.
- Validasi jadwal pickup PO (PREORDER_MIN_LEAD_MINUTES, minimal 30 menit, divalidasi di dua lapis: UI dan service layer) tidak boleh terpengaruh oleh refactor permission ini.

Scope

****Termasuk dalam scope:****

- Audit dan lengkapi seluruh 8 capability di MATRIX (`src/lib/permissions.ts`) dengan entry eksplisit untuk admin, owner, cashier, production. Tambahkan capability granular untuk akses halaman: `page.pos`, `page.shifts`, `page.pesanan`, dan capability lain yang saat ini masih dicek manual di kode (susuri seluruh codebase untuk pola `role === "admin"`, `role === "owner"`, atau pengecekan role manual serupa, dan konversi masing-masing menjadi capability MATRIX).
- Ganti `void role` di `pos-page.tsx` (saat ini role diambil dari `useRole()`) tapi tidak dipakai untuk guard apapun) menjadi guard aktif lewat `usePermission().can(...)`.
- Ganti pengecekan manual `role === "admin" || role === "owner"` di `shifts-page.tsx` menjadi guard lewat `usePermission().can(...)` yang membaca MATRIX yang sama.
- Tambah satu route guard generik di layout root (root layout component yang membungkus semua halaman ber-role) yang membaca dari MATRIX yang sama – ini mencegah user mengetik URL halaman secara langsung untuk bypass guard di level komponen halaman.

****Di luar scope:****

- Perubahan `navigation.ts` (item menu Production ke Pesanan) – itu Sprint 4.
- Perubahan business logic Shift packaging – itu Sprint 2.
- Perubahan logic Queue/Sale/refund – itu Sprint 3.
- Fitur baru apapun di luar permission enforcement.

Implementation Requirements

1. **Audit MATRIX** saat ini. Baca ``src/lib/permissions.ts`` secara utuh. Catat 8 capability yang sudah ada dan konfirmasi bahwa saat ini semuanya hanya berisi role ``"admin"``.
2. **Susuri codebase** untuk semua pengecekan role manual yang terpisah dari MATRIX – minimal yang sudah diketahui: ``pos-page.tsx`` (variabel role dari ``useRole()`` yang di-void), ``shifts-page.tsx`` (``role === "admin" || role === "owner"``). Cari pola serupa di file lain yang menyentuh Sales/POS/Shift/Pesanan.
3. **Definisikan capability baru** untuk setiap titik akses yang ditemukan di langkah 2, dengan penamaan konsisten (pola ``page.<nama>`` untuk akses halaman, ``action.<nama>`` untuk aksi spesifik seperti approve refund, buka shift, dll – sesuaikan dengan pola yang sudah dipakai di 8 capability existing MATRIX).
4. **Isi MATRIX dengan entry eksplisit** untuk keempat role di setiap capability (existing + baru). Owner = read-only di semua capability yang admin punya (tidak ada capability action/write kecuali yang sudah dikonfirmasi sebagai wewenang owner, misalnya set target dashboard – itu sudah benar dan jangan diubah). Cashier = capability operasional harian (POS, Shift buka/tutup, lihat Pesanan). Production = HANYA capability yang scope-nya Production & Inventory.
5. **Refactor titik pemakaian**: ganti setiap pengecekan role manual yang ditemukan di langkah 2 menjadi panggilan ``usePermission().can("<capability>")``, tanpa mengubah behavior fungsional lain di sekitar kode itu (jangan refactor logic lain yang tidak berhubungan dengan permission).
6. **Route guard generik**: implementasikan di layout root – komponen yang saat ini membungkus routing halaman-halaman ber-role. Guard ini harus redirect atau tampilkan halaman "akses ditolak" yang konsisten dengan UI existing kalau ``usePermission().can(...)`` untuk capability halaman yang sesuai return false. Guard ini harus membaca MATRIX yang sama, bukan duplikasi logic.
7. Pastikan tidak ada capability yang tertinggal hanya berisi ``"admin"`` tanpa entry eksplisit untuk role lain, kecuali capability tersebut memang secara bisnis hanya untuk admin (jika ragu, beri owner akses read-only sebagai default aman, dan cashier/production ``false`` secara eksplisit – bukan tertinggal kosong).

UI / UX Modernization

Sprint ini murni logic/security, tidak membuat UI baru kecuali halaman "akses ditolak" pada route guard (jika belum ada). Jika perlu membuat halaman/state baru itu:

- Ikuti hierarki visual, spacing, dan tipografi yang sudah dipakai di halaman lain aplikasi DeSalut (jangan bikin gaya baru).
- Pertahankan palet warna dan brand identity DeSalut yang sudah ada.
- Tampilkan pesan yang jelas dan sopan (Bahasa Indonesia) menjelaskan bahwa role user tidak punya akses ke halaman ini, tanpa membocorkan detail teknis capability internal.
- Sediakan tombol/link jelas untuk kembali ke halaman yang memang bisa

diakses oleh role tersebut.

Constraints

- JANGAN melakukan audit ulang – gunakan temuan di atas sebagai source of truth.
- JANGAN mengubah modul lain yang tidak disebutkan (Production, Inventory, Analytics) kecuali sekadar menambah capability baru di MATRIX untuk role production yang scope-nya memang Production & Inventory.
- JANGAN mengubah business rule apapun di luar permission (refund policy, validasi pickup PO, dll).
- JANGAN memperkenalkan dependency backend baru – semua tetap berjalan di atas localStorage seperti sekarang.
- JANGAN menghapus fungsionalitas yang sudah ada untuk role manapun kecuali memang itu adalah celah yang harus ditutup (contoh: cashier/production yang seharusnya tidak punya akses ke suatu capability).
- JANGAN meremake arsitektur file besar (`pos-page.tsx`, `shifts-page.tsx`) di luar titik permission – refactor structural sengaja ditunda sampai fitur stabil, ini keputusan yang sudah diambil sebelumnya dan harus dihormati.
- JANGAN mengubah `navigation.ts` di sprint ini.

Validation Checklist

- [] Semua 8 capability existing di MATRIX punya entry eksplisit untuk admin, owner, cashier, production (tidak ada yang hanya berisi admin).
- [] Semua capability baru yang ditemukan dari audit codebase sudah masuk MATRIX dengan entry eksplisit untuk keempat role.
- [] `pos-page.tsx` tidak lagi punya `void role` – role dipakai aktif untuk guard lewat `usePermission().can(...)`.
- [] `shifts-page.tsx` tidak lagi punya pengecekan manual `role === "admin" || role === "owner"` – sudah diganti ke `usePermission().can(...)`.
- [] Route guard generik di layout root berhasil memblokir akses halaman untuk role yang tidak punya capability-nya, dan mengizinkan akses untuk role yang punya.
- [] Cashier tetap bisa mengakses POS, Shift, dan Pesanan seperti sebelumnya (tidak ada regression fungsional untuk alur kasir harian).
- [] Owner bisa melihat (read-only) semua yang admin bisa lihat, tanpa bisa melakukan action yang sebelumnya terbatas ke admin.
- [] Production tidak lagi punya akses ke capability apapun di luar Production & Inventory.
- [] Kebijakan refund, validasi pickup PO 30 menit, dan seluruh business logic lain tidak berubah.
- [] Tidak ada error console/runtime baru muncul di halaman POS, Shift, dan Pesanan untuk keempat role.

Completion Criteria

Sprint ini selesai ketika: (1) MATRIX di `permissions.ts` sudah lengkap untuk seluruh capability existing dan baru, (2) tidak ada satu pun pengecekan role manual (`role === "...")` tersisa di `pos-page.tsx` dan `shifts-page.tsx`

yang seharusnya lewat MATRIX, (3) route guard generik aktif di layout root dan sudah diverifikasi memblokir/mengizinkan akses sesuai role dengan benar untuk keempat role di seluruh halaman Sales/POS/Shift/Pesanan, dan (4) seluruh item Validation Checklist di atas tercentang.

6. Complete Implementation Prompt — Sprint 2

SPRINT 2 – Packaging Auto-Prefill: Shift Management ↔ POS

Sprint Objective

Hilangkan duplikasi sumber data packaging antara POS (otomatis, akurat) dan Shift Management (manual, rawan human error). Ubah `PackagingUsageCard` di `shifts-page.tsx` supaya draft-nya di-prefill otomatis dari agregasi data pemakaian packaging yang sudah tercatat oleh `stockMovementService.record()` setiap kali ada checkout di POS, bukan mulai dari nol / `savedMap` saja.

Audit Findings Included

- Finding #1 (Critical): "Data Packaging di Shift Management masih manual, padahal data otomatis sudah ada di POS."

Business Rules to Preserve

- `sales.service.ts` sudah mencatat setiap pemakaian packaging otomatis lewat `stockMovementService.record()` dengan `type: "sales\_packaging\_usage"` setiap kali ada checkout di POS – logic pencatatan ini TIDAK berubah, sprint ini hanya menambah cara Shift Management MEMBACA data yang sudah tercatat itu.
- Kasir tetap harus bisa melakukan koreksi manual atas angka yang muncul (untuk kasus riil: packaging rusak/dibuang di luar transaksi) – prefill otomatis bukan berarti field menjadi read-only.
- Data yang direkonsiliasi harus dibatasi pada rentang waktu shift yang sedang aktif: dari `shift.openedAt` sampai waktu sekarang (saat card dibuka) – bukan seluruh histori.
- Pengelompokan data harus per `itemId` packaging, konsisten dengan struktur yang sudah dipakai di `savedMap` saat ini.

Scope

Termasuk dalam scope:

- Modifikasi titik inisialisasi draft di `PackagingUsageCard` (`shifts-page.tsx`) supaya, saat card dibuka, sistem memanggil `stockMovementService.list()`, memfilter hasilnya dengan `type === "sales\_packaging\_usage"` DAN `createdAt` berada di antara `shift.openedAt` hingga waktu sekarang, lalu mengagregasi total pemakaian per `itemId`.
- Hasil agregasi ini menjadi nilai awal draft (bukan lagi default 0 dari `savedMap` kosong).
- Kasir tetap bisa mengedit/mengoreksi angka yang sudah terisi otomatis

sebelum menyimpan – field tetap editable.

- Perilaku save/submit draft (menyimpan ke `savedMap` atau struktur penyimpanan yang sudah ada) tetap seperti sebelumnya – tidak ada perubahan pada mekanisme penyimpanan akhir, hanya pada nilai AWAL yang ditampilkan saat card dibuka.

****Di luar scope:****

- Perubahan struktur data `stockMovementService` itu sendiri (skema, tipe, fungsi `record()`) – service ini sudah benar, jangan disentuh.
- Perubahan alur checkout POS yang mencatat `sales\_packaging\_usage` – sudah benar, jangan disentuh.
- Fitur analitik/laporan packaging jangka panjang – itu domain modul Analytics, di luar scope modul Sales & POS ini sepenuhnya.
- Permission/guard untuk siapa yang boleh membuka `PackagingUsageCard` – itu domain Sprint 1, kalau ada gap di sana perbaiki di Sprint 1 bukan di sini.

Implementation Requirements

1. Baca `PackagingUsageCard` di `shifts-page.tsx` secara utuh untuk memahami struktur draft saat ini (bentuk state, cara `savedMap` diisi dan dibaca, cara field ditampilkan per item packaging).
2. Baca `stockMovementService.list()` di service layer untuk memahami signature, parameter filter yang tersedia, dan struktur objek yang dikembalikan (khususnya field `type`, `itemId`, `createdAt`, dan jumlah/quantity pemakaian).
3. Pada titik card dibuka (mount atau saat shift aktif terdeteksi), panggil `stockMovementService.list()`, filter di sisi client (atau lewat parameter filter kalau service sudah mendukung) untuk `type === "sales\_packaging\_usage"` dan `createdAt >= shift.openedAt`.
4. Agregasi (sum) quantity per `itemId` dari hasil filter tadi.
5. Gunakan hasil agregasi ini sebagai nilai INISIAL untuk state draft, menggantikan default 0 / nilai kosong dari `savedMap` yang sebelumnya jadi satu-satunya sumber.
6. Pastikan kalau kasir mengubah angka manual setelah prefill, perubahan itu tidak tertimpa ulang oleh re-fetch data POS (misalnya kalau card di-render ulang) – nilai draft harus tetap mengikuti input kasir setelah interaksi pertama, prefill hanya terjadi sekali saat card pertama dibuka/shift baru terdeteksi.
7. Pastikan performa: kalau `stockMovementService.list()` mengembalikan seluruh histori tanpa filter server-side, lakukan filtering yang efisien di memory (tidak masalah untuk skala data localStorage saat ini, tapi hindari loop bersarang yang tidak perlu).

UI / UX Modernization

`PackagingUsageCard` adalah komponen yang sudah ada dan user-facing (dilihat kasir setiap tutup/buka shift). Terapkan modernisasi ringan:

- Tambahkan indikator visual kecil (misalnya badge atau label halus) di sebelah angka yang terisi otomatis, menandai "otomatis dari POS" vs angka

yang sudah dikoreksi manual oleh kasir – ini membantu kasir langsung tahu mana yang perlu diverifikasi vs mana yang sudah mereka ubah sendiri.

- Pertahankan warna, tipografi, dan struktur card yang sudah ada – hanya tambahkan micro-interaction (misalnya transisi halus saat angka terisi otomatis) dan state visual yang jelas antara "belum diverifikasi" dan "sudah dikoreksi/dikonfirmasi kasir".
- Pastikan focus state dan hover state pada input field tetap konsisten dengan pola input lain di aplikasi.
- Jangan redesign total card – hanya poles kualitas visual dan tambahkan sinyal sumber data.

Constraints

- JANGAN mengubah ``sales.service.ts`` atau mekanisme pencatatan ``stockMovementService.record()``.
- JANGAN mengubah struktur penyimpanan akhir ``savedMap`` – hanya nilai inisial draft yang berubah.
- JANGAN menghapus kemampuan kasir untuk mengoreksi manual.
- JANGAN memperkenalkan backend/API baru – tetap `localStorage`.
- JANGAN menyentuh permission/guard shift (domain Sprint 1).
- JANGAN menyentuh logic Queue/Sale/refund (domain Sprint 3).

Validation Checklist

- [] Saat ``PackagingUsageCard`` dibuka untuk shift yang sedang aktif, draft terisi otomatis dari agregasi ``stockMovementService.list()`` yang difilter ``type === "sales_packaging_usage"`` dan rentang waktu shift yang benar.
- [] Agregasi dikelompokkan per ``itemId`` dengan benar, sesuai jumlah transaksi checkout POS yang sudah terjadi di shift tersebut.
- [] Kasir masih bisa mengedit angka yang sudah terisi otomatis.
- [] Perubahan manual kasir tidak tertimpa ulang oleh re-render/re-fetch.
- [] Untuk shift baru yang belum ada transaksi checkout sama sekali, draft tetap terisi 0 (bukan error).
- [] Tidak ada regresi pada mekanisme save/submit draft yang sudah ada sebelumnya.
- [] Tidak ada perubahan pada ``sales.service.ts`` atau ``stockMovementService``.

Completion Criteria

Sprint ini selesai ketika kasir yang membuka ``PackagingUsageCard`` di tengah atau akhir shift langsung melihat angka pemakaian packaging yang sudah terisi otomatis dan akurat sesuai transaksi checkout yang benar-benar terjadi di shift tersebut, dengan kemampuan koreksi manual tetap berfungsi, dan seluruh item Validation Checklist tercentang.

7. Complete Implementation Prompt — Sprint 3

SPRINT 3 – Relink Queue ↔ Sale saat Pre Order Complete (Fix Guard Refund Bocor)

Sprint Objective

Perbaiki bug integritas data di mana entri `Queue` yang dibuat dari Pre Order (`promoteDue()` di `preorder.service.ts`) tidak pernah di-relink `saleId`-nya dari placeholder `po.id` menjadi `sale.id` yang baru dibuat saat PO di-"Complete". Bug ini membuat SEMUA Pre Order yang di-Complete tampil salah status "Selesai" di halaman Pesanan (padahal dapur belum menyentuhnya), dan membuat guard refund (`canRefund()`) selalu meloloskan refund untuk sale asal Pre Order walau pesanan itu sudah masuk proses produksi.

Audit Findings Included

- Finding #3B (Critical, temuan tambahan pasca-audit final): "Sale hasil 'Complete' dari Pre Order tidak pernah relink ke Queue asal – guard refund bocor total."

Business Rules to Preserve

- Aturan bisnis "refund ditolak kalau pesanan sudah masuk proses produksi" HARUS berlaku untuk SEMUA sale, termasuk yang asalnya dari Pre Order – ini bukan aturan baru, ini aturan yang SUDAH ADA tapi bocor khusus untuk kasus Pre Order. Sprint ini menutup celahnya, bukan membuat aturan baru.

- Kebijakan refund tetap void seluruh transaksi (bukan partial per item) – tidak berubah oleh fix ini.

- `canRefund()` dan `deriveStatus()` di `pesanan-page.tsx` dan `sales.service.ts` TIDAK boleh diubah logic internalnya – begitu relink `saleId` benar, kedua fungsi ini otomatis bekerja benar tanpa modifikasi tambahan (ini eksplisit disebutkan di audit sebagai solusi yang diharapkan).

- Entri Queue yang sudah ada (lama) harus di-UPDATE (relink `saleId`), BUKAN dibuat entri Queue baru – status Queue lama (`waiting`/`cooking`/`dst`) harus tetap terjaga apa adanya, hanya `saleId` yang berubah.

Scope

****Termasuk dalam scope:****

- Modifikasi titik di mana PO di-"Complete" (fungsi yang memanggil `salesService.create()` dari `preorder-page.tsx`, sesuai deskripsi audit – konfirmasi nama fungsi persis dengan membaca kode).

- Tambahkan satu langkah: setelah `Sale` baru berhasil dibuat, cari entri Queue lama lewat `preorderId`/`po.queueId` (gunakan field yang memang sudah ada untuk menghubungkan Queue ke PO asalnya – baca `preorder.service.ts` dan skema Queue untuk memastikan field yang benar), lalu update field `saleId` pada entri Queue tersebut menjadi `sale.id` yang baru dibuat.

- Pastikan langkah relink ini terjadi SETELAH `Sale` berhasil dibuat dan SEBELUM proses "Complete" dianggap selesai secara keseluruhan (supaya tidak

ada window di mana Sale sudah ada tapi Queue masih menunjuk ke `po.id` lama).

- Verifikasi bahwa langkah existing `if (!sale.isPreorder) { queueService.create(...) }` TETAP seperti itu – untuk sale yang BUKAN dari Pre Order, perilaku pembuatan Queue baru tidak berubah sama sekali.

****Di luar scope:****

- Perubahan logic `canRefund()` atau `deriveStatus()` itu sendiri.
- Perubahan alur pembuatan Queue untuk sale non-Pre Order.
- Perubahan UI halaman Pesanan atau Preorder di luar efek otomatis dari fix status yang sekarang jadi benar.
- Fitur DP/Pelunasan – itu Sprint 5 (meski menyentuh fungsi yang sama, `complete()`, jangan gabungkan perubahan DP ke sprint ini; sprint ini HANYA fokus relink Queue).

Implementation Requirements

1. Baca `preorder.service.ts` untuk memahami `promoteDue()` – konfirmasi bahwa entri Queue yang dibuat memang menggunakan `saleId: po.id` sebagai placeholder, dan identifikasi field yang menghubungkan entri Queue itu kembali ke PO asalnya (`preorderId` atau `po.queueId`, cek nama field aktual di kode – JANGAN asumsikan nama field tanpa verifikasi langsung ke source).
2. Baca `preorder-page.tsx`, cari fungsi `complete()` (atau nama setara yang menjalankan proses PO → Sale). Pahami urutan operasi saat ini: `salesService.create()` dipanggil, lalu ada pengecekan `if (!sale.isPreorder) { queueService.create(...) }` yang secara sengaja SKIP pembuatan Queue baru untuk sale dari Pre Order.
3. Baca `queueService` untuk menemukan/konfirmasi fungsi update entri Queue by id atau by field pencarian (misalnya fungsi seperti `update()` atau `updateBySaleId()`/`updateById()` – gunakan fungsi yang sudah ada, jangan buat fungsi service baru kalau yang sudah ada bisa dipakai).
4. Tambahkan logic baru tepat setelah `Sale` baru dibuat di dalam `complete()`: cari entri Queue lama menggunakan field penghubung dari langkah 1 (bukan `saleId`, karena `saleId` di entri lama masih `po.id` – gunakan `preorderId`/`po.queueId` untuk pencarian), lalu panggil fungsi update Queue untuk mengganti field `saleId` entri tersebut menjadi `sale.id` yang baru.
5. Pastikan pencarian entri Queue lama robust terhadap kemungkinan entri tidak ditemukan (edge case: PO yang di-Complete tanpa pernah melalui `promoteDue()` – kalau ini secara bisnis tidak mungkin terjadi, cukup log/skip dengan aman tanpa melempar error yang menggagalkan proses Complete; JANGAN sampai proses Complete gagal total hanya karena entri Queue tidak ditemukan, karena itu akan lebih parah daripada bug asalnya).
6. Setelah relink diterapkan, verifikasi secara manual (baca ulang logic) bahwa `deriveStatus()` di `pesanan-page.tsx` (yang memakai `queueBySale.get(sale.id)`) sekarang akan menemukan entri Queue yang benar, dan `canRefund()` di `sales.service.ts` (yang memakai `queueService.bySale(id)`) juga akan menemukan entri yang sama – TANPA mengubah kode di kedua fungsi tersebut.

UI / UX Modernization

Sprint ini adalah data-integrity fix di service/logic layer, TIDAK ada perubahan UI baru yang diperlukan. Efek yang terlihat oleh user adalah status pesanan di halaman Pesanan yang sekarang menampilkan status Queue yang benar (bukan otomatis "Selesai") untuk sale asal Pre Order – ini adalah PERBAIKAN dari tampilan yang salah, bukan redesign. Jangan ubah tampilan visual halaman Pesanan atau Preorder di sprint ini.

Constraints

- JANGAN mengubah logic internal ``canRefund()`` atau ``deriveStatus()``.
- JANGAN membuat entri Queue baru untuk sale asal Pre Order – HARUS update entri lama, bukan create baru (kalau sampai create baru, akan ada dua entri Queue untuk satu proses produksi yang sama, membuat masalah baru).
- JANGAN mengubah perilaku pembuatan Queue untuk sale non-Pre Order.
- JANGAN menggabungkan perubahan fitur DP/Pelunasan ke dalam sprint ini meskipun menyentuh fungsi ``complete()`` yang sama – itu scope Sprint 5, harus jadi commit/prompt terpisah.
- JANGAN memperkenalkan backend/API baru.
- JANGAN mengasumsikan nama field koneksi Queue*PO tanpa memverifikasi langsung ke source code – audit menyebut ``preorderId`/`po.queueId`` sebagai kandidat, konfirmasi mana yang benar-benar dipakai di kode aktual.

Validation Checklist

- [] Saat PO dengan Queue berstatus ``waiting`` di-Complete, entri Queue tersebut TETAP berstatus ``waiting`` (tidak berubah status produksinya) tapi ``saleId``-nya sudah menunjuk ke ``sale.id`` yang baru, bukan lagi ``po.id``.
- [] ``deriveStatus()`` di halaman Pesanan untuk sale ini sekarang menampilkan status yang sesuai dengan status Queue yang sebenarnya (``waiting`/`cooking`/dst`), BUKAN otomatis "Selesai".
- [] ``canRefund()`` untuk sale ini sekarang mengembalikan hasil yang benar sesuai status produksi aktual – refund DITOLAK kalau Queue masih dalam proses produksi, DIIZINKAN kalau memang belum/sudah sesuai aturan bisnis yang berlaku untuk sale non-Pre Order.
- [] Sale non-Pre Order (yang sebelumnya sudah benar) TIDAK mengalami regresi – Queue baru tetap dibuat seperti biasa untuk sale jenis ini.
- [] Tidak ada duplikasi entri Queue untuk satu proses produksi yang sama setelah PO di-Complete.
- [] Proses Complete tidak gagal/error bahkan untuk edge case entri Queue lama tidak ditemukan.
- [] Tidak ada perubahan pada logic ``canRefund()`/`deriveStatus()`` itu sendiri – perbaikan seluruhnya terjadi di titik penulisan data (``complete()``), sesuai arahan audit.

Completion Criteria

Sprint ini selesai ketika setiap Pre Order yang di-Complete menghasilkan Sale baru yang entri Queue-nya sudah benar terhubung (relinked), status di halaman Pesanan mencerminkan status produksi yang sebenarnya, dan guard refund

menolak refund untuk sale asal Pre Order yang pesanannya sudah masuk proses produksi – sama seperti perilaku yang sudah benar untuk sale non-Pre Order. Seluruh item Validation Checklist tercentang.

8. Complete Implementation Prompt — Sprint 4

SPRINT 4 – Koreksi Akses Navigasi Role Production ke Pesanan

Sprint Objective

Tutup kontradiksi antara akses navigasi role Production dan keputusan bisnis yang sudah dikonfirmasi bahwa role Production seharusnya HANYA bisa mengakses modul Production dan Inventory. Saat ini `productionNav` di `navigation.ts` masih menyertakan item "Pesanan" yang mengarah ke `/sales/pesanan`.

⚠️ KEPUTUSAN BISNIS WAJIB DIAMBIL SEBELUM EKSEKUSI

Audit mencatat ini sebagai open question yang BELUM dijawab. Prompt ini menyediakan DUA opsi implementasi lengkap. AI coding assistant HARUS menghentikan eksekusi dan meminta konfirmasi opsi mana yang dipilih SEBELUM mulai menulis kode, kecuali instruksi tambahan di luar prompt ini sudah menyatakan opsi mana yang dipilih secara eksplisit.

- ****Opsi A (Easy):**** Hapus total item "Pesanan" dari `productionNav`.
Konsekuensi: tim produksi tidak lagi tahu antrean pesanan lewat menu ini sama sekali (mereka masih bisa bekerja dari modul Production yang sudah ada, tapi kehilangan visibility ke `/sales/pesanan`).
- ****Opsi B (Medium):**** Buat view antrean produksi berdiri sendiri di dalam modul Production (BUKAN reuse halaman `/sales/pesanan`), isinya HANYA "apa yang harus diproduksi" – tanpa akses ke data sales lain seperti harga atau detail customer lengkap.

Audit Findings Included

- Finding #3 (Critical): "Role Production masih bisa akses /sales/pesanan – kontradiksi dengan keputusan bisnis."

Business Rules to Preserve

- Role Production HANYA boleh mengakses modul Production dan Inventory – ini adalah keputusan bisnis yang SUDAH diambil dan dikonfirmasi, bukan sesuatu yang didiskusikan ulang di sprint ini.
- Kalau Opsi B dipilih: view antrean produksi baru TIDAK BOLEH menampilkan harga, detail customer lengkap (nama/no HP/catatan customer), riwayat pembayaran, atau data finansial apapun – hanya informasi yang murni dibutuhkan untuk memproduksi (misalnya: item apa, jumlah, kategori goreng/matang, catatan produk kalau ada catatan teknis produksi).
- Guard permission untuk halaman baru (kalau Opsi B) HARUS memakai MATRIX

dari Sprint 1 (``usePermission().can(...)``) – JANGAN membuat pengecekan role manual baru yang terpisah dari MATRIX (ini akan mengulang masalah yang baru saja diperbaiki di Sprint 1).

Scope

****Jika Opsi A dipilih – Termasuk dalam scope:****

- Hapus entry "Pesanan" dari ``productionNav`` di ``navigation.ts``.
- Verifikasi tidak ada link/shortcut lain di UI yang masih mengarahkan role production ke ``/sales/pesanan``.
- Pastikan kalau role production mencoba mengakses ``/sales/pesanan`` langsung lewat URL, route guard dari Sprint 1 sudah memblokirnya (kalau belum, ini adalah gap di Sprint 1 yang perlu diperbaiki, bukan invented requirement baru di sini – laporkan gap ini secara eksplisit kalau ditemukan).

****Jika Opsi B dipilih – Termasuk dalam scope:****

- Hapus entry "Pesanan" yang mengarah ke ``/sales/pesanan`` dari ``productionNav``.
- Tambah entry navigasi baru di ``productionNav`` yang mengarah ke halaman/route baru khusus antrean produksi (misalnya ``/production/antrean`` – sesuaikan pola routing yang sudah dipakai di modul Production).
- Buat komponen halaman baru yang membaca data Queue (lewat ``queueService``, service yang sudah ada dan sudah benar dipakai di modul lain) dan menampilkan HANYA: item yang harus diproduksi, jumlah, kategori goreng/matang, status Queue (waiting/cooking/dst), dan catatan produksi jika ada – TANPA field harga, customer, atau data finansial.
- Guard halaman baru ini dengan capability MATRIX baru (misal ``page.production-queue``) yang diberikan ke role production (dan admin/owner untuk visibility, sesuai pola akses read-only owner dari Sprint 1).

****Di luar scope (kedua opsi):****

- Perubahan pada ``queueService`` itu sendiri (kecuali menambah fungsi read baru yang scoped, kalau memang belum ada fungsi yang cukup granular – jangan ubah fungsi existing yang sudah dipakai modul lain).
- Perubahan alur POS/Pesanan untuk role selain production.
- Perubahan permission untuk role cashier/admin/owner – itu sudah selesai di Sprint 1.

Implementation Requirements

****Untuk Opsi A:****

1. Buka ``navigation.ts``, temukan array/objek ``productionNav``.
2. Hapus entry yang label-nya "Pesanan" dan path-nya ``/sales/pesanan``.
3. Jalankan pencarian codebase untuk referensi lain ke kombinasi role production + path ``/sales/pesanan`` (misalnya di breadcrumb, shortcut dashboard) dan hapus/sesuaikan jika ditemukan.
4. Verifikasi route guard Sprint 1 sudah cukup untuk memblokir akses langsung lewat URL – jika BELUM ada guard untuk ``/sales/pesanan`` yang scoped ke role, tandai ini sebagai temuan gap, JANGAN diam-diam menambah guard baru di luar

pola MATRIX Sprint 1.

****Untuk Opsi B:****

1. Buka `navigation.ts`, hapus entry "Pesanan" dari `productionNav` seperti Opsi A langkah 2.
2. Tentukan path route baru untuk antrean produksi, konsisten dengan pola URL modul Production yang sudah ada.
3. Buat komponen halaman baru yang mem-fetch data lewat `queueService` (gunakan fungsi read yang sudah ada, misalnya fungsi list/getAll – cek signature aktual di service).
4. Rancang tampilan data yang HANYA menunjukkan: nama item + jumlah, kategori goreng/matang, status Queue, dan catatan produksi kalau field itu memang ada dan relevan untuk produksi (bukan `customerNotes` yang sifatnya catatan pelanggan ke kasir).
5. Tambahkan entry navigasi baru ke `productionNav` mengarah ke path ini.
6. Tambahkan capability baru di MATRIX (Sprint 1) untuk halaman ini, berikan akses ke production (full) dan admin/owner (read-only, konsisten pola Sprint 1); cashier tidak perlu akses ke halaman ini kecuali sudah confirmed sebaliknya.
7. Terapkan route guard yang sama seperti pola Sprint 1 (guard generik di layout root harus otomatis meng-cover route baru ini kalau capability sudah didaftarkan dengan benar – verifikasi, jangan asumsikan).

UI / UX Modernization

****Untuk Opsi B (halaman baru dibuat):****

- Desain halaman antrean produksi dengan tampilan kartu/list yang jelas, terinspirasi Shopify POS/Linear untuk kejelasan status (gunakan badge warna berbeda untuk `waiting` vs `cooking` vs status lain yang ada).
- Pertahankan palet warna dan navigasi philosophy DeSalut yang sudah ada di modul Production.
- Prioritaskan kejelasan dan kecepatan baca untuk tim dapur yang bekerja cepat – ukuran teks besar untuk item & jumlah, hierarki visual yang jelas antara informasi penting (apa yang diproduksi) vs sekunder (catatan).
- Sediakan empty state yang jelas ("Tidak ada antrean produksi saat ini") kalau Queue kosong.
- Loading state yang jelas saat data di-fetch.

****Untuk Opsi A:**** tidak ada UI baru, tidak perlu modernisasi.

Constraints

- JANGAN mengeksekusi salah satu opsi tanpa konfirmasi eksplisit opsi mana yang dipilih.
- JANGAN menampilkan data finansial atau data customer lengkap di halaman antrean produksi (Opsi B).
- JANGAN membuat pengecekan role manual baru – semua guard HARUS lewat MATRIX Sprint 1.
- JANGAN mengubah `queueService` existing function signatures yang sudah dipakai modul lain.

- JANGAN memberi role production akses ke halaman `/sales/pesanan` dalam bentuk apapun, di opsi manapun.

Validation Checklist

- [] Konfirmasi opsi (A/B) sudah diterima secara eksplisit sebelum kode ditulis.
- [] Item "Pesanan" sudah hilang dari `productionNav`.
- [] Role production tidak lagi bisa mengakses `/sales/pesanan` baik lewat menu maupun URL langsung.
- [] (Opsi B saja) Halaman antrean produksi baru berhasil menampilkan data Queue yang benar, tanpa field harga/customer/finansial.
- [] (Opsi B saja) Guard capability halaman baru sudah terdaftar di MATRIX Sprint 1 dan berfungsi untuk keempat role sesuai definisi akses.
- [] Role selain production (admin/owner/cashier) tidak mengalami regresi akses ke `/sales/pesanan`.

Completion Criteria

Sprint ini selesai ketika role production tidak lagi memiliki jalur navigasi atau akses langsung ke `/sales/pesanan` dalam bentuk apapun, DAN (jika Opsi B) tim produksi tetap punya visibility ke apa yang harus diproduksi lewat halaman baru yang scope datanya sudah benar dibatasi. Seluruh item Validation Checklist tercentang.

9. Complete Implementation Prompt — Sprint 5

SPRINT 5 – DP & Pelunasan: Sambungkan UI + Fix Bug Double-Charge

Sprint Objective

Sambungkan service layer DP (Down Payment) yang sudah lengkap di `preorder.service.ts` (field `downPayment`, `dpMethod`, `remainingBalance`, `dpPaidAt`, plus audit log `preorder\_dp\_paid`) ke UI yang saat ini belum ada sama sekali. Tambahkan popup input DP opsional saat PO dibuat di POS, dan popup pelunasan berbasis `remainingBalance` saat PO di-"Complete". Perbaiki bug aktif di mana `complete()` di `preorder-page.tsx` memakai `total: subtotal` mentah – mengabaikan `remainingBalance` – yang menyebabkan customer yang sudah bayar DP akan ke-charge total penuh lagi (double-charge nyata).

Audit Findings Included

- Finding #5 (High): "Split Payment (b) – DP & Pelunasan: service layer sudah ada, tapi belum tersambung ke UI + ada bug double-charge aktif."

Business Rules to Preserve

- DP bersifat OPSIONAL saat membuat PO – kasir boleh kosongkan (tidak semua PO harus punya DP).

- Field `downPayment`, `dpMethod`, `remainingBalance`, `dpPaidAt` di service layer SUDAH BENAR dan SUDAH ADA – sprint ini tidak mengubah skema/fungsi service ini, hanya menyambungkannya ke UI.
- Audit log `preorder\_dp\_paid` yang sudah ada di service HARUS tetap tercatat setiap ada pembayaran DP – jangan bypass logging ini dari UI baru.
- Saat PO di-Complete, Sale yang terbentuk HARUS memakai `remainingBalance` (sisa yang belum dibayar), BUKAN `subtotal` penuh – ini fix untuk bug double-charge yang sedang aktif.
- Sprint ini TIDAK mengubah relink Queue↔Sale (sudah diperbaiki di Sprint 3)
- pastikan perubahan di `complete()` untuk sprint ini tidak menghapus/mengganggu logic relink yang sudah ditambahkan Sprint 3, keduanya harus berjalan bersamaan di dalam fungsi `complete()` yang sama.
- Validasi pickup PO (minimal 30 menit dari waktu pemesanan) tidak berubah oleh fitur DP ini.

Scope

****Termasuk dalam scope:****

- Popup/dialog opsional "Masukkan DP (boleh kosong)" yang muncul saat kasir membuat PO baru di `pos-page.tsx`. Popup ini memanggil field `downPayment`/`dpMethod` yang sudah didukung service – kasir input jumlah DP dan metode bayar DP (kalau ada), atau skip kalau tidak ada DP.
- Popup pelunasan di `preorder-page.tsx` saat kasir menekan "Complete" pada PO yang punya `remainingBalance` > 0 – popup ini menampilkan `remainingBalance` (bukan `total`/`subtotal`), kasir input pelunasan, baru status PO bisa berubah jadi Complete.
- Fix bug di fungsi `complete()`: ganti `total: subtotal` menjadi `total: po.remainingBalance ?? subtotal` (fallback ke subtotal HANYA kalau memang tidak ada DP sama sekali / `remainingBalance` tidak terdefinisi – bukan kondisi normal untuk PO yang punya DP).

****Di luar scope:****

- Perubahan skema/fungsi service `preorder.service.ts` (`downPayment`, `dpMethod`, `remainingBalance`, `dpPaidAt`, audit log `preorder\_dp\_paid`) – sudah lengkap dan benar, jangan disentuh.
- Split payment kombinasi metode bayar dalam satu transaksi (cash+QRIS sekaligus) – itu Sprint 7, domain berbeda meski konsepnya berdekatan ("payment"). JANGAN menggabungkan implementasi Sprint 7 ke sini.
- Perubahan logic relink Queue↔Sale – sudah selesai di Sprint 3, jangan diubah, hanya pastikan tidak rusak oleh perubahan sprint ini.
- Perubahan permission/guard – sudah selesai di Sprint 1, gunakan saja `usePermission().can(...)` yang sudah ada kalau popup baru butuh guard (misalnya hanya role tertentu yang boleh input DP – VERIFIKASI ke audit/stakeholder apakah ada batasan role untuk ini; kalau audit tidak menyebutkan batasan, asumsikan semua role yang sudah bisa akses POS/PO boleh input DP & pelunasan, konsisten dengan capability existing).

Implementation Requirements

1. Baca ``preorder.service.ts`` secara utuh untuk memahami signature fungsi yang sudah ada untuk mencatat DP (field-field yang disebutkan di atas serta fungsi yang menulis audit log ``preorder_dp_paid``) – JANGAN membuat fungsi service baru kalau fungsi yang dibutuhkan sudah ada.
2. Di ``pos-page.tsx``, temukan titik alur pembuatan PO (form/langkah di mana ``isPreorder: true`` di-set). Tambahkan UI popup/step opsional untuk input DP: jumlah (angka, boleh kosong/0) dan metode pembayaran DP (pakai daftar ``PaymentMethod`` yang sudah ada, jangan buat enum baru).
3. Pastikan data DP dari popup ini diteruskan ke fungsi service yang sesuai (dari langkah 1) saat PO disimpan, sehingga ``downPayment``, ``dpMethod``, ``dpPaidAt``, ``remainingBalance`` (dihitung dari ``subtotal - downPayment``) terisi dengan benar, dan audit log ``preorder_dp_paid`` tercatat.
4. Di ``preorder-page.tsx``, temukan fungsi ``complete()``. Sebelum status PO diubah jadi Complete, cek apakah ``po.remainingBalance`` > 0. Kalau ya, tampilkan popup pelunasan yang menunjukkan ``remainingBalance`` sebagai jumlah yang harus dibayar (BUKAN ``subtotal`` atau ``total`` awal), kasir input pembayaran pelunasan (metode bayar untuk pelunasan, boleh berbeda dari metode DP).
5. Setelah pelunasan dikonfirmasi (atau kalau ``remainingBalance`` sudah 0/tidak ada DP dari awal), lanjutkan proses ``complete()`` yang sudah ada – TERMASUK langkah relink Queue*Sale dari Sprint 3, yang harus tetap berjalan tanpa terganggu oleh perubahan ini.
6. Ganti baris yang membuat ``Sale`` baru: pastikan ``total`` Sale menggunakan ``po.remainingBalance ?? subtotal`` – bukan ``subtotal`` mentah. Field ini merepresentasikan sisa yang harus dibayar/dicatat sebagai penerimaan saat Complete, bukan nilai order penuh (yang sebagian sudah dibayar lewat DP sebelumnya).
7. Pastikan angka yang tercatat di shift closing/rekonsiliasi kasir untuk transaksi ini konsisten – total yang masuk ke Sale saat Complete adalah pelunasan (sisa), sedangkan DP yang sudah dibayar sebelumnya tercatat lewat audit log ``preorder_dp_paid`` terpisah (JANGAN sampai DP dihitung dua kali di laporan/rekonsiliasi – verifikasi titik mana saja yang membaca ``sale.total`` untuk shift closing dan pastikan itu tidak menghitung ulang DP yang sudah dicatat terpisah).

UI / UX Modernization

- Popup input DP saat buat PO: desain sebagai step opsional yang jelas ("boleh dikosongkan" harus terlihat jelas, bukan terkesan wajib) – gunakan pola dialog/modal yang konsisten dengan modal lain yang sudah ada di POS.
- Popup pelunasan saat Complete: tampilkan ``remainingBalance`` dengan tipografi yang jelas dan besar (ini adalah angka paling penting di popup ini), beserta ringkasan singkat (total order, DP yang sudah dibayar, sisa yang harus dibayar sekarang) – hierarki visual harus membuat kasir tidak bingung angka mana yang harus diinput.
- Gunakan badge/label kecil untuk menandai PO yang punya DP aktif di list PO (misalnya di ``preorder-page.tsx``), supaya kasir langsung tahu PO mana yang butuh popup pelunasan saat Complete.

- Pertahankan palet warna dan gaya dialog yang sudah dipakai di aplikasi – jangan bikin komponen dialog baru dari nol kalau sudah ada komponen dialog reusable.
- Tambahkan state loading/disabled pada tombol submit popup selama proses simpan berjalan, untuk mencegah double-submit (relevan juga untuk mencegah double-charge dari sisi UI, melengkapi fix di sisi logic).

Constraints

- JANGAN mengubah skema/fungsi `preorder.service.ts` yang sudah ada dan benar.
- JANGAN mengimplementasikan split payment kombinasi metode (cash+QRIS sekaligus) – itu scope Sprint 7 secara terpisah.
- JANGAN mengganggu/menghapus langkah relink Queue↔Sale dari Sprint 3 di dalam fungsi `complete()`.
- JANGAN membuat DP menjadi wajib – harus tetap opsional.
- JANGAN menghitung DP dua kali di manapun (service, UI, atau titik rekonsiliasi shift).
- JANGAN memperkenalkan backend/API baru.

Validation Checklist

- [] Kasir bisa membuat PO baru dengan ATAU tanpa DP – keduanya berfungsi tanpa error.
- [] Saat DP diinput, `downPayment`, `dpMethod`, `dpPaidAt` tercatat benar dan audit log `preorder\_dp\_paid` muncul.
- [] `remainingBalance` terhitung benar (`subtotal - downPayment`).
- [] Saat PO dengan `remainingBalance` > 0 di-Complete, popup pelunasan muncul menampilkan `remainingBalance` yang benar (bukan subtotal/total awal).
- [] Setelah pelunasan dikonfirmasi, `Sale` baru terbentuk dengan `total` = `remainingBalance` (bukan `subtotal` penuh) – bug double-charge sudah tidak terjadi.
- [] PO tanpa DP sama sekali (dari awal) tetap bisa di-Complete dengan `total` = `subtotal` (perilaku lama tetap benar untuk kasus ini).
- [] Relink Queue↔Sale dari Sprint 3 tetap berjalan benar untuk Sale yang terbentuk dari alur DP ini (status Pesanan & guard refund tetap akurat).
- [] Tidak ada penghitungan ganda DP di shift closing/rekonsiliasi.
- [] UI popup DP & pelunasan konsisten secara visual dengan komponen dialog lain di aplikasi.

Completion Criteria

Sprint ini selesai ketika kasir bisa mencatat DP opsional saat membuat PO, dan saat PO tersebut di-Complete, sistem meminta pelunasan berdasarkan `remainingBalance` yang benar – customer TIDAK PERNAH lagi ke-charge total penuh setelah sudah membayar DP. Seluruh item Validation Checklist tercentang.

10. Complete Implementation Prompt — Sprint 6

SPRINT 6 – Edge Case Murah: SALINAN, Auto-save Cart, Blokir Produk Nonaktif, Guard Shift Ganda

Sprint Objective

Tutup empat gap Medium priority yang masing-masing bereffort rendah dan independen satu sama lain: (1) label pembeda "SALINAN" pada struk reprint, (2) auto-save cart supaya tidak hilang saat browser di-refresh, (3) blokir eksplisit produk yang dihapus/nonaktif di cart saat checkout final, (4) guard untuk mencegah shift ganda aktif akibat double-klik tombol buka shift.

Audit Findings Included

- Finding #6 (Medium): "Struk reprint belum ada label pembeda 'SALINAN'."
- Finding #7 (Medium): "Cart hilang saat browser di-refresh."
- Finding #8 (Medium): "Produk yang dihapus/nonaktif di tengah cart di-skip diam-diam saat checkout."
- Finding #9 (Medium): "Belum ada guard untuk mencegah shift ganda aktif."

Business Rules to Preserve

- Struk asli (bukan reprint) TIDAK berubah tampilannya sama sekali – badge "SALINAN" HANYA muncul pada hasil reprint.
- Cart yang di-restore dari auto-save HARUS persis sama dengan state sebelum refresh (item, jumlah, kategori goreng/matang, catatan) – tidak ada data yang boleh hilang atau berubah.
- Produk yang diblokir saat checkout karena sudah dihapus/nonaktif HARUS diberi pesan yang jelas ke kasir (bukan silent-skip seperti sebelumnya) – kasir harus tahu ada masalah dan produk mana yang bermasalah.
- Guard shift ganda TIDAK mengubah alur normal buka shift untuk kasus yang benar (satu kasir, satu klik) – hanya mencegah race condition dari double-klik/multi-request bersamaan.

Scope

Termasuk dalam scope:

- Modifikasi `buildReceiptHtml()` di `invoice-receipt.ts`: tambah parameter `isReprint` (boolean), render badge "SALINAN" di dekat nomor order kalau `true`.
- Update titik pemanggilan `reprint()` di `pesanan-page.tsx` supaya memanggil `buildReceiptHtml()` dengan `isReprint: true`.
- Update titik pemanggilan cetak struk asli (saat checkout normal) supaya tetap memanggil dengan `isReprint: false` atau parameter di-omit dengan default `false`.
- Implementasi auto-save cart: setiap ada perubahan pada state cart di POS, simpan ke storage (localStorage, konsisten dengan pola storage yang sudah dipakai aplikasi ini) dengan debounce sekitar 1 detik.
- Implementasi restore cart: saat halaman POS dibuka/dimuat ulang, cek apakah

ada cart tersimpan yang belum checkout, kalau ada – restore otomatis ke state cart.

- Di titik checkout final (sebelum `salesService.create()` dipanggil atau titik validasi terakhir sebelum submit), tambahkan pengecekan: untuk setiap item di cart, verifikasi produk masih ada dan aktif (bukan di-skip diam-diam). Kalau ada produk yang sudah dihapus/nonaktif, BLOKIR proses checkout dan tampilkan pesan jelas ke kasir menyebutkan produk mana yang bermasalah.
- Tambahkan guard di `shiftService.open()` (atau titik pemanggilannya di UI) untuk mencegah pemanggilan ganda yang menghasilkan dua shift aktif bersamaan – bisa lewat disable tombol setelah klik pertama sampai response selesai, DAN/ATAU pengecekan di level service bahwa tidak ada shift aktif lain sebelum membuat shift baru.

****Di luar scope:****

- Perubahan desain struk secara keseluruhan di luar penambahan badge "SALINAN".
- Fitur cart yang lebih kompleks (multi-cart, cart tersimpan lintas device) – hanya persistence sederhana untuk mencegah kehilangan data akibat refresh di device yang sama.
- Perubahan logic penentuan produk aktif/nonaktif itu sendiri (field/flag yang menandai produk nonaktif) – gunakan yang sudah ada.
- Race condition multi-device/multi-tab (Finding #10) – SENGAJA TIDAK dikerjakan di sprint ini maupun sprint manapun dalam roadmap ini, sesuai keputusan audit untuk diturunkan ke Low dan dipantau saja.

Implementation Requirements

****#6 – Badge SALINAN:****

1. Baca `buildReceiptHtml()` di `invoice-receipt.ts`, pahami struktur HTML yang dihasilkan dan di mana posisi nomor order dirender.
2. Tambah parameter `isReprint: boolean = false` ke signature fungsi.
3. Kalau `isReprint === true`, sisipkan elemen badge/label "SALINAN" di dekat nomor order – gunakan styling yang jelas terlihat tapi tidak merusak layout struk (misalnya border atau background halus, teks kapital, ukuran proporsional dengan elemen struk lain).
4. Update SEMUA titik pemanggilan `buildReceiptHtml()` di codebase – pastikan titik reprint mengirim `true`, titik cetak asli mengirim `false`/default.

****#7 – Auto-save cart:****

1. Identifikasi state cart di `pos-page.tsx` (struktur data item cart yang sudah ada).
2. Tambahkan effect/listener yang men-debounce (± 1 detik) setiap perubahan cart dan menyimpannya ke localStorage dengan key yang jelas (misal `desalut\_pos\_cart\_draft`).
3. Saat komponen POS mount, cek apakah ada draft tersimpan; kalau ada dan belum expired/checkout, restore ke state cart.
4. Setelah checkout berhasil (transaksi selesai), HAPUS draft cart tersimpan – supaya tidak ke-restore lagi di sesi berikutnya untuk transaksi yang sudah

selesai.

****#8 – Blokir produk nonaktif di cart:****

1. Temukan titik validasi terakhir sebelum submit checkout di `pos-page.tsx`.
2. Untuk setiap item di cart, verifikasi ulang status produk (aktif/tidak, masih ada/dihapus) terhadap data produk terkini (bukan snapshot saat produk ditambahkan ke cart).
3. Kalau ditemukan produk bermasalah, batalkan proses submit, tampilkan pesan error yang jelas menyebutkan nama produk yang bermasalah, dan biarkan kasir memutuskan (hapus item dari cart secara manual, dsb) – TIDAK ADA auto-skip diam-diam.

****#9 – Guard shift ganda:****

1. Baca `shiftService.open()` untuk memahami mekanisme pembuatan shift baru saat ini.
2. Tambahkan pengecekan di awal fungsi: kalau sudah ada shift aktif (belum ditutup) untuk konteks yang sama, tolak pembuatan shift baru dan return/throw kondisi yang jelas.
3. Di sisi UI (titik tombol "Buka Shift"), disable tombol segera setelah diklik pertama kali sampai response dari `shiftService.open()` diterima, untuk mencegah double-klik mengirim dua request hampir bersamaan.

UI / UX Modernization

- Badge "SALINAN": gunakan warna yang cukup kontras untuk terlihat jelas di struk cetak/preview, tapi tetap dalam palet yang konsisten dengan brand DeSalut (hindari warna asing yang tidak pernah dipakai di elemen lain).
- Pesan error blokir produk nonaktif: gunakan komponen alert/toast yang sudah ada di aplikasi (jangan `alert()` browser native), dengan bahasa yang jelas dan actionable untuk kasir (Bahasa Indonesia, sopan, langsung ke poin).
- Tombol "Buka Shift" saat disabled/loading: tampilkan indikator loading yang jelas (spinner kecil atau teks "Membuka...") supaya kasir tidak mengira tombol tidak merespons dan mengklik berulang.
- Restore cart: opsional tapi disarankan – tampilkan notifikasi halus ("Cart sebelumnya berhasil dipulihkan") saat restore terjadi, supaya kasir tidak bingung melihat cart terisi tanpa mereka input sendiri.

Constraints

- JANGAN mengubah desain struk asli di luar penambahan badge SALINAN untuk reprint.
- JANGAN membuat sistem cart yang lebih kompleks dari yang diminta (single draft per device sudah cukup, jangan bikin multi-draft/history cart).
- JANGAN mengubah field/flag yang menandai produk aktif/nonaktif – hanya membaca dan memvalidasi field yang sudah ada.
- JANGAN mengerjakan Finding #10 (race condition multi-device) – di luar scope roadmap ini sepenuhnya untuk saat ini.
- JANGAN memperkenalkan backend/API baru.

Validation Checklist

- [] Struk asli (cetak pertama saat checkout) TIDAK memiliki badge SALINAN.
- [] Struk hasil reprint SELALU memiliki badge SALINAN yang jelas terlihat.
- [] Cart yang sedang diisi tetap ada setelah browser di-refresh (item, jumlah, kategori, catatan sama persis).
- [] Setelah checkout berhasil, draft cart tersimpan terhapus (tidak ke-restore lagi untuk transaksi yang sudah selesai).
- [] Checkout dengan produk yang sudah dihapus/nonaktif di cart DIBLOKIR dengan pesan jelas, bukan silent-skip.
- [] Kasir bisa melanjutkan checkout setelah menghapus/mengganti item yang bermasalah.
- [] Double-klik tombol "Buka Shift" tidak menghasilkan dua shift aktif – hanya satu shift yang berhasil dibuat.
- [] Alur normal buka shift (satu klik, satu kasir) tetap berjalan tanpa perubahan pengalaman.

Completion Criteria

Sprint ini selesai ketika keempat finding Medium ini tertutup secara independen tanpa saling mengganggu – struk reprint punya label pembeda, cart persisten terhadap refresh, produk nonaktif diblokir eksplisit saat checkout, dan tidak ada lagi risiko shift ganda dari double-klik. Seluruh item Validation Checklist tercentang.

11. Complete Implementation Prompt — Sprint 7

SPRINT 7 – Split Payment: Kombinasi Metode Bayar dalam Satu Transaksi

Sprint Objective

Implementasikan kemampuan mencatat transaksi dengan gabungan metode bayar (misalnya sebagian cash, sebagian QRIS) dalam satu transaksi POS. Ubah `Sale.payment` dari tipe single value (`PaymentMethod`) menjadi array `{method, amount}[ ]`, update form checkout POS untuk input multi-metode, dan update SEMUA titik yang membaca `sale.payment` (shift closing/rekonsiliasi, receipt, analytics) supaya kompatibel dengan struktur baru.

⚠️ KEPUTUSAN BISNIS WAJIB DIAMBIL SEBELUM EKSEKUSI

Audit mencatat open question yang BELUM dijawab: saat kombinasi metode bayar dipakai, apakah total dari semua metode harus PAS sama dengan tagihan (tidak boleh lebih/kurang sama sekali), ATAU tetap ada logic "kembalian" yang berlaku khusus di porsi cash (misalnya kasir terima cash lebih besar dari porsi yang seharusnya, sistem hitung kembalian dari situ, sementara porsi metode lain seperti QRIS harus pas). AI coding assistant HARUS menghentikan eksekusi dan meminta konfirmasi jawaban ini SEBELUM menulis kode, kecuali instruksi tambahan di luar prompt ini sudah menjawabnya secara eksplisit.

Bagian "Implementation Requirements" di bawah menyediakan langkah untuk KEDUA kemungkinan jawaban – pilih jalur yang sesuai dengan konfirmasi yang diterima.

Audit Findings Included

- Finding #4 (High): "Split Payment (a) – kombinasi metode bayar dalam satu transaksi: belum ada sama sekali."

Business Rules to Preserve

- Kebijakan refund (void seluruh transaksi, bukan partial per item) tidak berubah – refund untuk transaksi split payment tetap membatalkan SELURUH transaksi, tidak ada refund partial per metode bayar.
- Marketplace/source channel (GoFood, Shopee, WA, Outlet) tidak berubah oleh perubahan struktur payment ini – kedua konsep ini independen (source channel = dari mana order berasal, payment method = bagaimana dibayar).
- Fitur DP & Pelunasan dari Sprint 5 HARUS tetap berfungsi dengan benar setelah perubahan tipe data ini – `dpMethod` (metode bayar untuk DP) dan metode bayar untuk pelunasan boleh tetap single method (TIDAK wajib mendukung split payment untuk DP/pelunasan kecuali dikonfirmasi sebaliknya – audit tidak menyebutkan kebutuhan split untuk DP, jangan invented requirement baru di sini).
- Semua Sale LAMA yang sudah tersimpan dengan `payment` sebagai single value HARUS tetap terbaca dengan benar oleh kode baru (backward compatibility) – jangan sampai data lama menyebabkan error saat dibaca ulang.

Scope

****Termasuk dalam scope:****

- Ubah tipe `Sale.payment` di `types.ts` dari `PaymentMethod` menjadi `{method: PaymentMethod, amount: number}[ ]`.
- Update form checkout di `pos-page.tsx`: tambahkan kemampuan kasir memasukkan lebih dari satu metode bayar dengan jumlah masing-masing, sebelum transaksi bisa disubmit.
- Validasi total gabungan sesuai keputusan bisnis yang sudah dikonfirmasi (lihat bagian keputusan wajib di atas).
- Update logic shift closing/rekonsiliasi supaya menghitung total per metode bayar dengan benar dari array `payment[ ]` (bukan lagi single value), termasuk breakdown per metode untuk keperluan rekonsiliasi kasir.
- Update `buildReceiptHtml()` (`invoice-receipt.ts`) supaya menampilkan breakdown metode bayar dengan jelas di struk kalau lebih dari satu metode dipakai (misalnya "Cash: Rp X, QRIS: Rp Y").
- Update titik-titik di modul Analytics yang membaca `sale.payment` (kalau modul Sales & POS punya kewajiban expose data ini ke Analytics lewat interface yang sudah ada – sesuaikan dengan cara Analytics saat ini mengonsumsi data Sales, JANGAN membuat interface baru kalau yang sudah ada bisa diadaptasi).
- Migrasi/kompatibilitas data lama: pastikan Sale lama dengan `payment` sebagai single value tetap terbaca benar (misalnya lewat normalisasi saat

baca: kalau `payment` bukan array, treat sebagai array berisi satu entry dengan `amount` = `total` Sale tersebut).

****Di luar scope:****

- Perubahan pada DP/Pelunasan (Sprint 5) untuk mendukung split payment – di luar scope kecuali dikonfirmasi terpisah.
- Perubahan kebijakan refund partial per metode bayar – TETAP void seluruh transaksi, tidak ada refund partial.
- Perubahan struktur data marketplace/source channel.
- Redesain modul Analytics – hanya menyesuaikan titik konsumsi data `sale.payment` yang sudah ada, bukan membangun fitur analytics baru.

Implementation Requirements

1. Baca `types.ts`, temukan definisi `Sale` dan `PaymentMethod` saat ini. Ubah tipe `payment` pada `Sale` menjadi `{method: PaymentMethod; amount: number}[]`.
2. Susuri SELURUH codebase untuk setiap titik yang membaca `sale.payment` sebagai single value (perkiraan area: checkout submission, shift closing/rekonsiliasi, `invoice-receipt.ts`, titik ekspos data ke Analytics) – daftar semua titik ini sebelum mulai mengubah kode, supaya tidak ada yang terlewat.
3. Di `pos-page.tsx`, ubah form checkout: tambahkan UI untuk kasir menambah lebih dari satu baris metode bayar (pilih metode + input jumlah), dengan kemampuan menambah/menghapus baris metode.
4. ****Jika keputusan bisnis = "harus pas 100%":**** validasi bahwa jumlah seluruh `amount` di array `payment[]` harus SAMA PERSIS dengan total tagihan sebelum tombol submit aktif – tidak ada logic kembalian sama sekali untuk kombinasi metode (kembalian, jika ada, tetap berlaku HANYA untuk transaksi single-method cash seperti perilaku lama, karena kombinasi metode secara definisi tidak butuh kembalian kalau harus pas).
5. ****Jika keputusan bisnis = "tetap ada logic kembalian di porsi cash":**** izinkan kasir memasukkan jumlah cash yang lebih besar dari porsi yang seharusnya, hitung kembalian dari SELISIH cash yang diinput dikurangi (total tagihan dikurangi jumlah metode non-cash yang sudah diinput), sementara metode non-cash (QRIS, dst) harus tetap pas sesuai porsi yang diinput kasir tanpa logic kembalian.
6. Simpan `payment[]` final ke `Sale` saat `salesService.create()` dipanggil.
7. Update logic shift closing: agregasi total per metode bayar dari SELURUH Sale dalam shift, dengan setiap Sale yang split payment dihitung kontribusinya ke MASING-MASING metode sesuai `amount` di array-nya (bukan seluruh nilai Sale masuk ke satu metode).
8. Update `buildReceiptHtml()`: kalau `payment.length > 1`, render breakdown per metode dengan jelas di struk; kalau `payment.length === 1`, tampilan tetap seperti sebelumnya (single method, tidak perlu breakdown berlebihan untuk kasus umum).
9. Tambahkan normalisasi baca data lama: di titik mana pun `sale.payment` dibaca, kalau ternyata bukan array (data lama), bungkus jadi `[method:

`sale.payment, amount: sale.total}]`` sebelum diproses lebih lanjut – pertimbangkan menaruh normalisasi ini di satu tempat terpusat (misalnya helper function) supaya tidak duplikasi logic normalisasi di banyak titik konsumsi.

UI / UX Modernization

- Form input multi-metode bayar: desain sebagai list baris yang bisa ditambah/dihapus, dengan total berjalan yang jelas terlihat (menampilkan sisa yang belum tercover / kelebihan, sesuai keputusan bisnis kembalian) – pola ini umum di POS modern seperti Shopify POS, ikuti kejelasan visual serupa.
- Highlight visual yang jelas kalau total belum pas (misalnya border merah halus + teks sisa yang harus diinput) supaya kasir tidak salah submit.
- Breakdown metode bayar di struk: format rapi, sejajar, mudah dibaca (nama metode kiri, jumlah kanan, konsisten dengan alignment item lain di struk).
- Rekonsiliasi shift closing dengan breakdown per metode: tampilkan sebagai daftar/tabel kecil yang jelas per metode bayar, konsisten dengan gaya card/summary yang sudah dipakai di halaman shift.
- Pertahankan palet warna dan navigasi philosophy DeSalut – perubahan ini menambah kompleksitas form, jadi prioritaskan kejelasan hierarki visual di atas estetika baru.

Constraints

- JANGAN mengubah kebijakan refund menjadi partial per metode bayar – tetap void seluruh transaksi.
- JANGAN mengeksekusi implementasi tanpa keputusan bisnis soal logic kembalian/pas-total dikonfirmasi lebih dulu.
- JANGAN merusak kompatibilitas baca data Sale lama (single-method) – WAJIB ada normalisasi.
- JANGAN mengubah fitur DP/Pelunasan dari Sprint 5 untuk mendukung split payment kecuali dikonfirmasi terpisah sebagai kebutuhan baru.
- JANGAN memperkenalkan backend/API baru – tetap localStorage.
- JANGAN melewati satu pun titik konsumsi ``sale.payment`` yang sudah didaftarkan di langkah audit codebase – regresi paling besar dari sprint ini datang dari titik yang terlewat.

Validation Checklist

- [] Konfirmasi keputusan bisnis (pas 100% vs logic kembalian di cash) sudah diterima sebelum implementasi dimulai.
- [] ``Sale.payment`` di ``types.ts`` sudah berubah menjadi array ``{method, amount}[]``.
- [] Kasir bisa checkout dengan satu metode bayar (perilaku lama tetap berfungsi, tidak ada regresi).
- [] Kasir bisa checkout dengan kombinasi lebih dari satu metode bayar, sesuai logic validasi yang sudah dikonfirmasi.
- [] Shift closing/rekonsiliasi menampilkan breakdown per metode bayar yang benar, termasuk kontribusi dari transaksi split payment.
- [] Struk transaksi split payment menampilkan breakdown metode bayar yang

jelas; struk transaksi single-method tampil seperti sebelumnya.

- [] Data Sale lama (single-method, format lama) tetap terbaca dan terhitung benar di semua titik konsumsi (shift closing, receipt reprint, analytics) tanpa error.
- [] Refund untuk transaksi split payment tetap membatalkan seluruh transaksi (bukan partial per metode).
- [] Fitur DP & Pelunasan (Sprint 5) tetap berfungsi normal tanpa regresi setelah perubahan tipe data ini.
- [] Tidak ada titik konsumsi `sale.payment` yang terlewat (error/undefined behavior) di seluruh codebase.

Completion Criteria

Sprint ini selesai ketika kasir bisa mencatat transaksi dengan kombinasi metode bayar sesuai aturan bisnis yang sudah dikonfirmasi, seluruh titik konsumsi data payment (shift closing, receipt, analytics) sudah update dan akurat, data lama tetap kompatibel, dan tidak ada regresi pada fitur lain (refund, DP/Pelunasan) yang bersinggungan dengan payment. Seluruh item Validation Checklist tercentang.

12. Catatan Penutup

- Dua open question dari audit (Bagian 5) — logic kembalian split payment (Sprint 7) dan opsi akses navigasi Production (Sprint 4) — **sengaja tidak diasumsikan** di dokumen ini. Kedua prompt sprint tersebut menyertakan blok keputusan eksplisit yang mewajibkan konfirmasi sebelum AI coding assistant mulai menulis kode, sesuai prinsip audit sebagai satu-satunya source of truth.
- Semua item yang secara eksplisit dikeluarkan dari roadmap oleh audit (Bagian 2 dokumen audit) — produk favorit, diskon, quick order, pajak/NPWP, partial refund per item, Sales History halaman terpisah, dashboard kompleks, halaman Kelola Kategori terpisah — **tidak muncul di sprint manapun** dan tidak akan muncul kecuali ada audit/keputusan bisnis baru yang mencabut pengecualian tersebut.
- Finding #10 (race condition multi-device/multi-tab) sengaja **tidak dijadikan sprint** — statusnya tetap "dipantau", sesuai keputusan audit bahwa ini hanya perlu dieksekusi begitu ada backend dengan session tracking server-side.
- Setiap prompt sprint di atas ditulis untuk bisa langsung ditempel ke sesi AI coding assistant baru tanpa konteks tambahan — masing-masing sudah membawa Business Rules, Scope, dan Constraints yang relevan secara mandiri.